

Music Knowledge Graph

Intelligent Semantic System with Automated Inference

Course: Semantic Web 2025/26

Technologies: Python • Django • RDF • GraphDB • SPARQL • React

Dataset: 30 000 Spotify Songs from Kaggle

Group 5

[112793] Cláudia Seabra

[129359] Nihal Fateen

June 2nd, 2026

1. Introduction to the topic.....	3
2. Definition of the ontology (RDFS and OWL).....	3
2.1. Base classes.....	3
2.2. Inferred/Defined Classes	3
2.3. Properties and OWL Structure	4
2.4. Protégé Verification	4
3. Set of Inferences (SPIN).....	5
3.1. Implemented Rules.....	5
4. New Operations on the Data (SPARQL)	5
4.1. Inferred Knowledge Extraction	6
4.2. Dynamic Dataset Expansion	6
4.3. Macro-level Interfaces Control.....	6
4.4. Automated Graph Maintenance	6
4.5. Operations	6
4.5.1. Read Operations	6
4.5.2. Update Operations	7
5. Use and Integration of Data from Wikidata and DBpedia.....	7
5.1. Dynamic Enrichment Architecture	7
5.2. Generated Enrichment Artifacts	7
6. Publication of Semantic Data through RDFa and microformats.....	8
6.1. Vocabulary Selection and Ontological Mapping	8
6.2. React Component Integration Strategy	8
6.2.1. Artist Pages	8
6.2.2. Album and Track Pages.....	8
7. New Application Functionalities (UI)	9
7.1. Inferred Knowledge Visualization	9
7.2. Context-Aware Federation Indicators.....	9
7.3. System Functionalities Summary	9
7.3.1. Frontend Features.....	9
7.3.2. Backend Features	10
8. Conclusions.....	10
9. Configuration and Execution.....	11
9.1. Data Repository	11
9.2. Backend Setup	11
9.3. Frontend Setup	11
9.4. SPIN Generation and Execution	11
9.5. Smoke Checks.....	11
9.6. Verification Evidence	12
Appendix: Deliverable File Map	12

1. Introduction to the topic

The Music Knowledge Graph (MusicKG) project transforms the Spotify playlist dataset into a semantic web application. Instead of treating artists, albums, tracks, genres, and audio features as isolated rows, the system represents them as RDF resources connected through a formal music ontology.

The first version of the project focused mainly on generating and exploring RDF data. The second phase extends that foundation into an intelligent semantic system: it defines an RDFS/OWL ontology, imports the graph into GraphDB, executes SPARQL queries and updates, materializes SPIN-compatible rules, links local resources to DBpedia/Wikidata, and publishes semantic annotations in the React frontend.

The dataset is generated from `spotify_songs.csv`. The current runtime evidence reports approximately 10 000 artists, 21 000 albums, 27 000 tracks, and 762 000 RDF triples when the backend is connected to the GraphDB repository `music-kg`.

The central goal of TP2 is not only to store RDF, but to demonstrate that the graph can be validated, queried, updated, enriched, inferred over, and exposed through a user interface. The project therefore combines backend semantic operations with a frontend that makes the resulting knowledge visible to a user.

2. Definition of the ontology (RDFS and OWL)

The ontology is exported to `music_kg_project/data/ontology.ttl` and uses the `http://musickg.org/ontology#` namespace. It describes the domain vocabulary used by the generated facts, the backend SPARQL queries, and the semantic annotations surfaced in the frontend.

2.1. Base classes

- `music:Artist` represents a music creator or group and is aligned with `schema:MusicGroup`.
- `music:Album` represents a collection of tracks released as a single project and is aligned with `schema:MusicAlbum`.
- `music:Track` represents an individual music recording and is aligned with `schema:MusicRecording`.
- `music:Genre` represents a stylistic category (e.g., pop, rock, edm) linked to artists and tracks and is aligned with `schema:MusicGenre`.
- `music:AudioFeatures` stores normalized Spotify acoustic metrics such as energy, danceability, valence, tempo, and loudness.
- `music:AudioProfile` is a derived profile used by rule-based classification over track characteristics.

2.2. Inferred/Defined Classes

- `music:HighEnergyTrack` is Inferred for any `music:Track` linked to a `music:AudioFeatures` node where the `music:energy` float value is > 0.8 .

- `music:PopularTrack` is used for tracks whose popularity score passes the popularity threshold.
- `music:ReleaseEra` represents broad temporal groupings such as Classic Era, Transition Era, and Modern Era.
- `music:ModernTrack` and `music:ClassicTrack` classify tracks according to the release year of their album.

2.3. Properties and OWL Structure

- `music:hasAlbum`: Connects an `music:Artist` (Domain) to an `music:Album` (Range).
- `music:hasTrack` & `music:onAlbum` (Inverse Properties): `music:hasTrack` connects an Album to a Track.
- `music:performedBy`: Connects a `music:Track` (Domain) to the respective `music:Artist` (Range).
- `music:performs` is declared as the inverse of `music:performedBy`.
- `music:similarTo` is declared as a symmetric artist-to-artist relation.
- `music:belongsToEra` links an album or track to a release era resource.
- `music:hasAudioFeatures` and `music:hasAudioProfile` link tracks to acoustic feature nodes.

Datatype properties include `artistName`, `albumName`, `trackName`, `releaseYear`, `popularity`, `durationMs`, `energy`, `danceability`, `valence`, `tempo`, and `loudness`.

2.4. Protégé Verification

The ontology-only file *ontology.ttl* was opened successfully in Protégé. Protégé displayed 190 axioms, 70 logical axioms, 16 classes, 13 object properties, 14 data properties, and 3 individuals.

The HermiT 1.4.3.456 reasoner was selected and started successfully. Protégé reported the reasoner as active. This confirms that the ontology is syntactically valid and can be processed by an OWL reasoner.

The integrated RDF/XML file *music_kg_integrated.rdf* also opened successfully in Protégé. This file combines ontology definitions with generated facts and classification triples, allowing inspection of individuals as well as the schema.

The screenshot displays the Protégé interface for the 'Music Knowledge Graph Ontology'. The main workspace shows the ontology header with the IRI 'http://musickg.org/ontology' and version '1.0.0'. The 'Ontology metrics' panel on the right provides a detailed breakdown of the ontology's structure, including counts for axioms, classes, and properties. The 'Class axioms' and 'Object property axioms' sections further detail the logical relationships defined in the ontology. The bottom of the interface shows the 'Imported ontologies' and 'Direct imports' sections.

Ontology metrics	
Axiom	761,597
Logical axiom count	761,471
Declaration axioms count	38
Class count	16
Object property count	13
Data property count	14
Individual count	94,717
Annotation Property count	2

Class axioms	
SubClassOf	9
EquivalentClasses	0
DisjointClasses	1
GCI count	0
Hidden GCI Count	0

Object property axioms	
SubObjectPropertyOf	0
EquivalentObjectProperties	0
InverseObjectProperties	1
DisjointObjectProperties	0
FunctionalObjectProperty	0
InverseFunctionalObjectProperty	0

3. Set of Inferences (SPIN)

While standard Description Logic (DL) reasoners like Hermit handle basic structural hierarchies (e.g., transitive and inverse properties), they are inherently limited when dealing with complex mathematics and cross-entity aggregations. To resolve this, a set of custom inference rules was developed using SPARQL Inferencing Notation (SPIN) principles.

SPIN-compatible inference rules are defined in *music_kg_project/music_graph/spin_rules.py*. The module has two purposes: it generates a SPIN-compatible Turtle deliverable and it exposes equivalent executable SPARQL UPDATE operations for GraphDB through a Django management command.

```
python3 music_kg_project/music_graph/spin_rules.py --output
music_kg_project/data/spin_rules.ttl
cd music_kg_project && python manage.py run_spin
```

The current implementation contains five SPIN-compatible rules. These rules express knowledge that is not always explicit in the source CSV. For example, release-era relation is derived from a year value, and symmetric similarity links avoid needing to store both directions manually. These rules also complement OWL reasoning.

The generated file *music_kg_project/data/spin_rules.ttl* is included as a TP2 deliverable, while the management command provides a reproducible way to run the same inference logic over the active RDF store.

3.1. Implemented Rules

- Infer performances from albums and tracks: If an artist has an album and the album has a track, the rule inserts `music:performs` between the artist and the track.
- Infer symmetric similarTo links: If artist A is similar to artist B, the rule inserts the reverse relation when it is missing.
- Classify high-energy tracks: Tracks with an audio profile energy value of at least 0.75 are typed as `music:HighEnergyTrack` and receive a high energy label.
- Classify popular tracks: Tracks with popularity of at least 70 are typed as `music:PopularTrack` and receive a high popularity label.
- Classify release eras: Albums and tracks are linked to Classic, Transition, or Modern era resources based on release year, and tracks are typed as `music:ModernTrack` or `music:ClassicTrack` where applicable.

4. New Operations on the Data (SPARQL)

In the first project, the application primarily utilized standard SPARQL SELECT queries to retrieve basic graph topologies (e.g., fetching an artist's discography or executing the hybrid text search). To support the Intelligent Semantic System developed in this phase, the operational architecture (*sparql_queries.py* and *linked_data.py*) was fundamentally upgraded. The system now executes complex analytical queries and performs real-time graph modifications, transitioning the triplestore from a static database into a dynamic, read-write repository.

A notable implementation detail is route-safe slug handling. The backend normalizes encoded artist and album slugs so resources such as `/api/artists/%21deladap/` resolve correctly to the encoded

RDF URI <http://musickg.org/artist/%21deladap>.

4.1. Inferred Knowledge Extraction

Since the SPIN inference engine automatically assigns multiple classes to entities (e.g., a single node might simultaneously be a `music:Artist`, `owl:NamedIndividual`, `music:LegendaryArtist`, and `music:MainstreamArtist`), the standard retrieval queries had to be modified to isolate the newly inferred intelligence.

To achieve this, the `get_artist_detail` operation was updated with a specific filtering pattern. The query retrieves the `rdf:type` of the entity but strictly filters out the base ontology classes, returning only the reasoner-inferred classes to be displayed as UI badges.

4.2. Dynamic Dataset Expansion

The most significant operational upgrade in this project is the programmatic modification of the live GraphDB repository. When the combined engine fetches an abstract from DBpedia, the system caches this data in memory and permanently materializes it into the local graph.

This is achieved by parameterized INSERT DATA queries executed via the GraphDB update endpoint. The query binds the dynamically fetched and sanitized English abstract string (`?safe_abstract`) to the local artist URI using a custom datatype property (`music:dbpediaAbstract`):

4.3. Macro-level Interfaces Control

To control the new macro-level interfaces (the Analytics and Timeline pages), the backend executes new SPARQL queries. For example, to generate the Genre Landscape Analytics, the system groups all tracks by their respective genre node, counts the total volume, and calculates the mathematical average of specific audio features. This requires the use of nested graph patterns and aggregate functions (AVG, COUNT).

4.4. Automated Graph Maintenance

If a user utilizes the frontend UI to move or delete a track, the system must ensure no "orphan" nodes are left behind. A specialized SPARQL DELETE / WHERE query utilizing the NOT EXISTS filter was created to execute immediately after any track modification. This query scans the graph for `music:Album` nodes that no longer possess any incoming `music:onAlbum` relationships and safely purges them.

4.5. Operations

4.5.1. Read Operations

- GET `/api/stats/` returns dataset statistics and indicates whether the backend is using GraphDB or rdflib fallback.
- GET `/api/artists/` and GET `/api/artists/<slug>/` return artist lists and artist detail data.
- GET `/api/tracks/` returns track-level results.
- GET `/api/search/` provides UI search over artists, albums, and tracks.
- GET `/api/timeline/`, `/api/genre-landscape/`, `/api/audio-distribution/`, and `/api/similar-edges/` support analytic views.
- POST `/api/sparql/` provides a controlled SELECT query endpoint.

4.5.2. Update Operations

- POST /api/artists/create/ creates a new artist node and genre link.
- POST /api/songs/bulk-create/ inserts multiple tracks for an artist and creates related album/track triples.
- POST /api/tracks/update-album/ moves a track to another album.
- POST /api/albums/update-year/ updates release-year data using SPARQL DELETE/INSERT logic.
- POST /api/tracks/delete/ removes track triples and cleans references.
- POST /api/sparql/update/ supports development/admin SPARQL UPDATE demonstrations.

5. Use and Integration of Data from Wikidata and DBpedia

To guarantee that the application is interoperable, we developed a Dynamic Federated Enrichment Engine using the Python SPARQLWrapper library (*music_kg_project/miusic_graph/linked_data.py*). This engine actively queries DBpedia to fetch missing contextual information, specifically artist biographies, and permanently merging them into the local graph. For example, the project generates owl:sameAs links from local artist names to DBpedia resources, then uses these links to enrich the local graph.

5.1. Dynamic Enrichment Architecture

Rather than executing a massive batch script to enrich all 10 316 artists simultaneously, the system employs a strategy tied to user interaction via the Django backend (*views.py*).

When a user navigates to an artist page (ArtistDetail) on the frontend, the backend intercepts the request and performs a local graph check:

1. It verifies if the local `music:Artist` node possesses an owl:sameAs link to a valid DBpedia URI.
2. It checks if the local graph already contains a `music:dbpediaAbstract` for this artist.
3. If the DBpedia URI exists but the abstract is missing, it triggers the federated `fetch_and_save_dbpedia_data()` function *synchronously* before returning the HTTP response to the client.
4. Returned values are cleaned and inserted into the local RDF graph using properties such as `music:dbpediaAbstract`, `music:originPlace`, and `music:officialWebsite`.
5. When DBpedia exposes a Wikidata entity, the project records `music:wikidataEntity` and an additional owl:sameAs Wikidata link.

5.2. Generated Enrichment Artifacts

```
music_kg_project/data/enrichment.ttl  
music_kg_project/data/music_kg_enriched.ttl
```

The latest recorded enrichment run queried 20 candidates, enriched 9 resources, added 25 triples, and completed with 0 failures. This provides concrete evidence that the integration is functional and not just theoretical.

The frontend also benefits from this linked data. Artist detail pages can expose a DBpedia link and publish the same external identity through schema.org sameAs attributes in the rendered HTML.

6. Publication of Semantic Data through RDFa and microformats

The React frontend publishes semantic annotations directly in rendered pages using RDFa-style attributes and schema.org microdata, so that the semantic meaning is visible not only inside GraphDB but also in the application HTML.

6.1. Vocabulary Selection and Ontological Mapping

The frontend data was mapped to Schema.org, the industry-standard vocabulary supported by major search engines (Google, Bing, Yahoo).

The mapping between the backend ontology and the frontend Schema.org presentation was defined as follows:

- `music:Artist` → `schema:MusicGroup`
- `music:Album` → `schema:MusicAlbum`
- `music:Track` → `schema:MusicRecording`
- **Datatype Properties** → `schema:name`, `schema:genre`, `chema:datePublished`
- `owl:sameAs` → `schema:sameAs`

6.2. React Component Integration Strategy

We used RDFa attributes (vocab, typeof, property) because they integrate seamlessly into React's component-based architecture.

6.2.1. Artist Pages

- `ArtistDetailPage.jsx` declares `vocab="https://schema.org/"` and `typeof="MusicGroup"`.
- The artist name is annotated with `property="name"` and `itemProp="name"`.
- Genres are annotated with `property="genre"`.
- External identity is exposed through `property="sameAs"` and `itemProp="sameAs"` when a DBpedia URI exists.

6.2.2. Album and Track Pages

- `AlbumDetailPage.jsx` declares `typeof="MusicAlbum"` and `itemType="https://schema.org/MusicAlbum"`.
- Album title, artist, and release year are mapped to schema.org properties including `name`, `byArtist`, and `datePublished`.
- Track listings are nested as `typeof="MusicRecording"` resources and expose track names through `property="name"`.
- The structure mirrors the backend graph: artists have albums, albums have tracks, and tracks are music recordings.

7. New Application Functionalities (UI)

Two major complementary UI functionalities were introduced to surface this semantic intelligence directly to the user.

7.1. Inferred Knowledge Visualization

A primary challenge in developing automated reasoning (SPIN rules) is that the resulting inferences are often invisible to the end-user. If the ontology automatically classifies an artist as a `music:LegendaryArtist` due to their extensive discography, this knowledge remains trapped in the triplestore unless explicitly rendered.

To resolve this, the Artist Profile interface was upgraded to visually distinguish between explicit raw data (derived from the original CSV) and machine-inferred knowledge (generated by the Reasoner).

- The frontend parses the `inferred_classes` array provided by the updated SPARQL SELECT query. Standard data, such as generic genres (e.g., "Pop", "Rap"), are rendered in their traditional UI styling. Conversely, inferred ontological classes (e.g., `LegendaryArtist`, `MainstreamArtist`, `IndieArtist`, `HighEnergyTrack`) are mapped to highly distinct visual elements.

7.2. Context-Aware Federation Indicators

To prevent the user from perceiving the delay from the API response of the live DBpedia SPARQL endpoint as an application freeze, a Context-Aware Loading Indicator was implemented using React's `useEffect` hooks and a delayed state timeout architecture.

- When an artist profile is requested, the frontend initiates an 800-millisecond timer. If the API resolves before the timer finishes (indicating the data was served locally from GraphDB), the UI renders immediately without interruption. However, if the promise remains unresolved after 800ms (indicating the backend is currently paused to fetch and ingest data from DBpedia), a specialized, non-blocking toast notification appears reading: *"Fetching latest data from DBpedia..."*. The moment the external data is saved and returned, the notification is instantly dismissed.

7.3. System Functionalities Summary

7.3.1. Frontend Features

- Home page with dataset statistics loaded from the backend.
- Search page for artists, albums, and tracks with filters and sorting.
- Artist detail page with genres, albums, tracks, audio feature summaries, DBpedia links, and similar artists.
- Album detail page with RDFa/schema.org annotation and nested tracks.
- Graph explorer showing artist, genre, and similarity relationships.
- Timeline view over release years and genre evolution.
- Analytics pages for genre landscape and audio distribution.
- Recommendations based on audio-feature similarity and genre overlap.

7.3.2. Backend Features

- GraphDB-first RDFStore with rdflib fallback when GraphDB is unavailable.
- SPARQL SELECT and SPARQL UPDATE support.
- Similarity engine built from artist audio feature vectors.
- Search index built from spotify_songs.csv for responsive UI search.
- Linked-data enrichment using DBpedia and Wikidata.
- Management command for running SPIN-compatible inference updates.

8. Conclusions

The second phase of the Music Knowledge Graph successfully elevated a static RDF database into a dynamic system. By rigorously applying OWL constraints and programmatic SPIN rules, the system proved capable of classifying entities and identifying patterns that standard SQL relational databases cannot autonomously achieve.

The programmatic integration with DBpedia ensures the dataset is continuously expanding, while the implementation of Schema.org RDFa ensures this knowledge is accessible to the broader Semantic Web. The ontology is used by RDF generation, GraphDB, SPARQL queries, inference rules, API responses, and UI rendering.

The project also demonstrates practical verification. *ontology.ttl* and *music_kg_integrated.rdf* were opened in Protégé, HermiT was started successfully, GraphDB smoke testing returned HTTP 200 in GraphDB mode, the frontend build completed, and browser UI smoke tests passed.

The most significant limitation is that the Django project currently has no unit tests, so *manage.py* test reports 0 project tests. However, this is mitigated by system checks, API smoke testing, frontend build verification, and browser workflow testing.

9. Configuration and Execution

9.1. Data Repository

- Ensure GraphDB is running on `http://localhost:7200`. The repository ID used by the backend is `music-kg`.
- Create a new repository and manually import the files needed to support the environment.
- Recommended import order: *facts_only.nt*, *ontology.ttl*, and *spin_rules.ttl*. The integrated `music_kg_integrated.rdf` file is used mainly for Protégé validation and inspection.

9.2. Backend Setup

1 — Create and activate a Python 3 virtual environment.

```
python3 -m venv .venv
source .venv/bin/activate
```

2 — Install dependencies: `pip install -r requirements.txt`

```
cd music_kg_project
pip install -r requirements.txt
```

3 — Start the API server (Runs on port 8000).

```
python manage.py migrate
python manage.py runserver
```

9.3. Frontend Setup

1 — Open a new terminal and navigate to the `music-kg-frontend` directory.

```
cd music-kg-frontend
```

2 — Install packages

```
npm install
```

3 — Start the frontend server

```
npm run dev
```

4 — Visit `http://localhost:3000` to access application.

9.4. SPIN Generation and Execution

```
python3 music_kg_project/music_graph/spin_rules.py --output
music_kg_project/data/spin_rules.ttl
cd music_kg_project && python manage.py run_spin
```

9.5. Smoke Checks

```
curl http://localhost:8000/api/stats/
```

The expected GraphDB-mode response includes `backend="GraphDB"`, `graphdb_url="http://localhost:7200"`, and `repository="music-kg"`.

9.6. Verification Evidence

- Protégé opened *ontology.ttl* successfully; HermiT 1.4.3.456 was started and reported as active.
- *music_kg_integrated.rdf* opened successfully in Protégé.
- GraphDB smoke test: GET `/api/stats/` returned HTTP 200 with backend set to GraphDB and repository `music-kg`.
- Frontend production build completed successfully with Vite.
- Browser smoke test covered Home, Search, Timeline, Analytics, Graph, search for Drake, and artist detail navigation without console errors.
- Encoded artist route check passed for both `/api/artists/%21deladap/` and `/api/artists/%2521deladap/`.

Appendix: Deliverable File Map

- *facts_only.nt*: generated instance data without ontology definitions.
- *ontology.ttl*: RDFS/OWL ontology and semantic vocabulary.
- *music_kg_integrated.rdf*: ontology plus generated facts for Protégé inspection.
- *spin_rules.ttl*: generated SPIN-compatible Turtle rules.
- *enrichment.ttl*: linked-data enrichment triples from DBpedia/Wikidata.
- *music_kg_enriched.ttl*: enriched RDF output.
- *stats.json*: generated statistics consumed by the backend and UI.

. Each listed file corresponds to a concrete TP2 deliverable or verification artifact in the project folder.